

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JNANA SANGAMA”, BELAGAVI - 590018



**DEEP LEARNING MINI PROJECT REPORT(CS722T2C) on
“Sentiment Analysis of Product Reviews”**

Submitted by

Sumangala Gouda

4SF23CS220

Sushmitha S

4SF24CS421

Sanjanna KS

4SF23CS192

In partial fulfillment of the requirements for the V semester

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE & ENGINEERING

Under the Guidance of

Ms. Shiji Abraham

Assistant Professor

Department of CSE

at



SAHYADRI

College of Engineering & Management

An Autonomous Institution

MANGALURU

2026- 27



SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

EVALUATION RUBRICS

Name: Sumangala Gouda

USN:4SF23CS220

Sl. No.	Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Poor (1)	Marks
1	Problem Definition & Objectives	Problem is clearly defined, relevant, and objectives are specific and measurable	Problem and objectives are clear with minor gaps	Problem is partially defined; objectives need improvement	Problem/objectives are unclear	
2	Technical Approach & Implementation	Appropriate methodology, algorithms/tools used effectively; implementation is complete	Appropriate technical approach with minor issues	Basic implementation with limited technical depth	Incomplete or inappropriate implementation	
3	Innovation / Application / Problem Solving	Demonstrates strong originality, creativity, and effective solution to the identified problem	Shows some innovation and good problem-solving	Limited innovation; mostly conventional approach	No significant innovation/problem-solving	
4	Results, Testing & Analysis	Results are clearly presented, thoroughly tested, and critically analyzed using suitable measures	Results presented with adequate testing and analysis	Basic results and limited testing/analysis	Results are incomplete or unsupported	
5	Documentation, Presentation & Viva	Well-structured report, clear presentation, excellent communication and confident viva	Good documentation, presentation and satisfactory viva	Adequate documentation/presentation with limited clarity	Poor documentation, presentation or viva	
	Total					

S I . N O .	Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Poor (1)	Mark s
1	Problem Definition & Objectives	Problem is clearly defined, relevant, and objectives are specific and measurable	Problem and objectives are clear with minor gaps	Problem is partially defined; objectives need improvement	Problem/objectives are unclear	
2	Technical Approach & Implementatio n	Appropriate methodology, algorithms/tools used effectively; implementation is complete	Appropriate technical approach with minor issues	Basic implementation with limited technical depth	Incomplete or inappropriate implementation	
3	Innovation / Application / Problem Solving	Demonstrates strong originality, creativity, and effective solution to the identified problem	Shows some innovation and good problem-solving	Limited innovation; mostly conventional approach	No significant innovation/problem-solving	
4	Results, Testing & Analysis	Results are clearly presented, thoroughly tested, and critically analyzed using suitable measures	Results presented with adequate testing and analysis	Basic results and limited testing/analysis	Results are incomplete or unsupported	
5	Documentatio n, Presentation & Viva	Well-structured report, clear presentation, excellent communication and confident viva	Good documentation, presentation and satisfactory viva	Adequate documentation/presentation with limited clarity	Poor documentation, presentation or viva	
	Total					

Name: Sanjanna KS

USN: 4SF23CS193

S I . N o .	Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Poor (1)	Mark s
1	Problem Definition & Objectives	Problem is clearly defined, relevant, and objectives are specific and measurable	Problem and objectives are clear with minor gaps	Problem is partially defined; objectives need improvement	Problem/objectives are unclear	
2	Technical Approach & Implementatio n	Appropriate methodology, algorithms/tool s used effectively; implementatio n is complete	Appropriate technical approach with minor issues	Basic implementation with limited technical depth	Incomplete or inappropriate implementation	
3	Innovation / Application / Problem Solving	Demonstrates strong originality, creativity, and effective solution to the identified problem	Shows some innovation and good problem- solving	Limited innovation; mostly conventional approach	No significant innovation/proble m-solving	
4	Results, Testing & Analysis	Results are clearly presented, thoroughly tested, and critically analyzed using suitable measures	Results presented with adequate testing and analysis	Basic results and limited testing/analysis	Results are incomplete or unsupported	
5	Documentatio n, Presentation & Viva	Well-structured report, clear presentation, excellent communication and confident viva	Good documentatio n, presentation and satisfactory viva	Adequate documentation/presentatio n with limited clarity	Poor documentation, presentation or viva	
	Total					



SAHYADRI
COLLEGE OF ENGINEERING & MANAGEMENT
An Autonomous Institution
MANGALURU

Department of Computer Science & Engineering

DECLARATION

We hereby declare that the entire work embodied in this Mini Project Report titled “**Project Title**” has been carried out by us at Sahyadri College of Engineering & Management, Mangaluru under the supervision of **Ms. Shiji Abraham**, in partial fulfillment of the requirements for the VII semester of **Bachelor of Engineering in Computer Science & Engineering**. This report has not been submitted to this or any other University for the award of any other degree.

Sumangala Gouda(4SF23CS220)

Sushmitha S(4SF24CS421)

Sanjanna KS(4SF23CS193)

Dept. of CSE, SCEM, Mangaluru

Abstract

Understanding whether a piece of written text carries a positive or negative tone is a common challenge in natural language processing, and this project tackles that challenge in the context of movie reviews. The work delivers a web-based Movie Review Sentiment Analyzer capable of judging, in real time, whether a submitted review leans favourable or unfavourable. At its heart lies a Bidirectional LSTM (BiLSTM) network, developed using the PyTorch framework and trained on a benchmark collection of 50,000 labelled IMDb reviews. Before any prediction can be made, the raw review text goes through a cleaning and tokenisation stage and is converted into fixed-length padded sequences; these sequences are then fed into an embedding layer followed by the BiLSTM, which reads the sentence in both forward and backward directions to build a richer contextual understanding of the wording. The resulting representation is passed through a fully connected layer with a sigmoid activation, yielding both a predicted sentiment label and an associated confidence value. Rather than remaining a standalone model, the trained network is wrapped inside a multi-page Streamlit web application, giving users a simple interface to type in a review and immediately see how the model has classified it. Each prediction made through the app is automatically stored in an SQLite database, and a dedicated dashboard section presents this accumulated data through visual trends and summary statistics, along with options to filter, search, and export the records as a CSV file. On the IMDb test set, the model reaches an accuracy of roughly 83%, a result that underlines how effective BiLSTM architectures can be for opinion-mining tasks, while also pointing toward transformer-based techniques as a promising avenue for further improving performance.

Table of Contents

Content	Page No.
Abstract	i
Table of Contents	ii
1. Introduction 1.1 Overview 1.2 Scope & Motivation	1
2. Literature Survey (Refer at least 20 Most recent suitable papers)	1
3. Problem Formulation 3.1 Problem Description 3.2 Problem Statement 3.3 Objectives 3.4 Functional Requirements 3.5 Non-Functional Requirements	3
4. Project Design and Implementation 4.1 Proposed Project Architecture 4.2 High Level Design 4.3 Detailed Design 4.4 Datasets Descriptions 4.5 Tools and Technologies used for Implementation	5
5. Results and Discussion 5.1 Experimentation Environment set up 5.2 Functional Requirements Results achieved (Tabular & Graphical results with Descriptions) 5.3 Non-Functional Requirements Results achieved (Tabular & Graphical results with Descriptions) 5.4 Comparison of Results with Existing works (Cited in Literature Survey) 5.5 Project GUI Snapshots (Major Functionalities) 5.6 Societal Impact of the Project 5.7 SDG Mapped	9
Conclusion and Future scope	19
References	19

Introduction

1.1. Overview

People constantly share opinions online — about movies, products, services — on platforms like IMDb, Amazon, and Twitter. Buried in this text is an emotional tone, positive or negative, that sentiment analysis, a branch of NLP, tries to detect automatically. Older methods used fixed word lists or basic models like Naive Bayes and SVM, but these missed word order and negation, often confusing "good" with "not good." Deep learning changed this — LSTM, and especially its two-way version BiLSTM, can follow how meaning builds across a sentence, giving far better results. This project focuses on movie reviews using the IMDb dataset, since it's large, free, and practically useful for platforms that want to know if a review is favourable. The result is a complete system: a BiLSTM model trained on IMDb data, wrapped in a preprocessing pipeline, and deployed as a Streamlit web app with prediction logging and a dashboard.

1.2. Scope and Motivation

This project sticks to one task-classifying English movie reviews as positive or negative using a BiLSTM model trained on 50,000 IMDb reviews. It covers the full pipeline, from raw text to a working web app, but leaves out star-rating prediction, aspect-based analysis, and other languages as future work. Two things motivated this project. Practically, reading thousands of reviews by hand isn't realistic, so an automated classifier saves effort and can support recommendation tools or review dashboards. Personally, it offered hands-on experience with the full deep-learning workflow-preparing data, building a model in PyTorch, training it, and deploying it- much like real-world NLP work.

2. Literature Survey

S.No.	Author(s) & Year	Focus Area	Key Contribution / Finding
1	ScienceDirect (2025)	RoBERTa + CNN (R-CNN)	Built a web app using RoBERTa and CNN together to check movie review sentiment.
2	Greeshma & Simon (2024)	BiGRU + Attention	Used BiGRU with GloVe and attention; got better results than plain BiGRU/BiLSTM.
3	Krishna et al. (2024)	Data Augmentation + Pooling	Added synonym-based data boosting and pooling to make IMDb sentiment analysis more efficient
4	Palomo et al. (2024)	Transformer Models BERT/RoBERTa/XLNet	Tested BERT, RoBERTa, XLNet on IMDb; transformers beat older TF-IDF methods but cost more compute.
5	Atandoh et al. (2024)	Scalable DL Framework (PEW-MCAB)	Built a deep learning model that reads full review text, not just word bags, for better sentiment prediction.
6	NCBI/PMC (2024)	BiLSTM / BiLSTM-CNN (Drug Reviews)	Used BiLSTM and BiLSTM-CNN on drug reviews; got 87–96% accuracy, showing BiLSTM works outside movies too.
7	Gill et al. (2023)	CNN/LSTMBi LSTM+Self Attention	Compared CNN, LSTM, BiLSTM with attention; BiLSTM with attention worked best.
8	Singh & Singla (2023)	Comparative ML/DL Study	Tested six models on IMDb; BiLSTM gave the highest accuracy among all.
9	Ansari et al., arXiv:2304.08369 (2023)	Word2Vec vs BERT	Compared Word2Vec and BERT for social media sentiment; BERT performed better on tricky text.

10	arXiv:2311.11250 (2023)	Survey of Sentiment Analysis	Reviewed different sentiment analysis methods.
11	Jiang, Hu & Jiang (2022)	Word2Vec + LSTM	Combined Word2Vec with LSTM for movie reviews and got strong results with a simple setup.
12	Chakraborty et al., arXiv:2104.10662 (2021)	Word Embedding	Reviewed bag-of-words, TF-IDF, Word2Vec and GloVe techniques.
13	Hameed & GarciaZapirain (2020)	Single-Layer BiLSTM	Showed a shallow BiLSTM with careful pre-processing achieves competitive accuracy.
14	Devlin et al. (2019)	BERT	Introduced BERT; a strong pre-trained model but needs high computing power.
15	Rezaeinia et al. (2017)	IWV	Improved word vectors by adding grammar and sentiment info; boosted accuracy.
16	Giatsoglou et al. (2017)	Hybrid Embeddings	Mixed word embeddings with sentiment lexicons for better classification.
17	Vaswani et al. (2017)	Transformer / Attention	Introduced the Transformer model using only attention, no recurrence needed.
18	Pennington, Socher & Manning (2014)	GloVe Embeddings	Introduced GloVe embeddings, built from word co-occurrence patterns.
19	Kim (2014)	CNN for Text Classification	Applied CNNs to sentence classification, capturing local n-gram patterns with fewer parameters than recurrent models.
20	Mikolov et al. (2013)	Word2Vec Embeddings	Proposed CBOW and Skip-gram models for learning dense word vectors, motivating learned embedding layers.

3. Problem Formulation

3.1. Problem Description

Movie review sites collect huge amounts of free-text opinions every day, but this text rarely comes with a clean positive/negative tag that can be used for analysis. Star ratings, when they exist at all, are often inconsistent or missing entirely. Going through this text by hand to judge sentiment is slow, subjective, and simply can't keep up with how much content gets posted daily. What's needed is a system that can read a review written in plain language and work out, on its own, whether it's positive or negative - along with some indication of how confident it is in that judgment.

3.2. Problem Statement

To build a deep-learning system that takes a movie review written in English and predicts whether its sentiment is Positive or Negative, along with a confidence score, and to make this available through a web app that also keeps a record of past predictions and displays them visually.

3.3. Objectives

- Gather and clean the IMDb Large Movie Review Dataset (50,000 labelled reviews) to use for a binary sentiment-classification task.
- Design and build a Bidirectional LSTM (BiLSTM) model in PyTorch that can pick up on sentiment patterns across a sequence of words.
- Train the model and check its performance using standard metrics — accuracy, precision, recall, and F1-score.
- Build a preprocessing and encoding pipeline that behaves exactly the same way during inference as it did during training, so predictions stay consistent.
- Build a multi-page Streamlit app where a user can type in a review and see its predicted sentiment right away.
- Save every prediction to an SQLite database and add a dashboard for viewing trends, filtering, searching, and exporting this history as a CSV file.

3.4. Functional Requirements

- **Review Input** - Users can type or paste a movie review (up to 500 characters) into a text box on the web page.
- **Text Cleaning** - The system cleans and breaks down the input text the same way it was done while training the model.
- **Text Encoding** - The cleaned text is converted into a fixed-length sequence of vocabulary IDs that the model can process.
- **Sentiment Prediction** - The system labels the review as "Positive" or "Negative" and shows a confidence score along with it.
- **Prediction Logging** - Every prediction (review text, label, confidence, and timestamp) gets saved to a database automatically.
- **Dashboard & Analytics** - A dashboard shows overall stats (total, positive, and negative counts), a pie chart, and a daily trend line.
- **Search, Filter & Export** - Users can filter results by sentiment, search by keyword, and download the full history as a CSV file.
- **Sample Reviews** - A few pre-loaded example reviews let users try the tool instantly without typing anything.

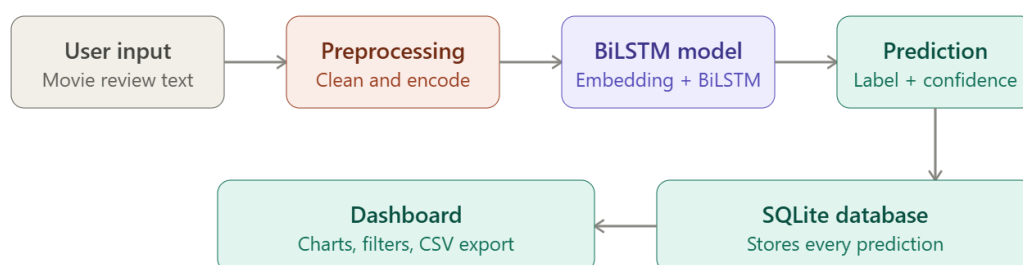
3.5. Non-functional Requirements

- **Performance:** predictions should come back within a few seconds on an ordinary CPU.
- **Usability:** the interface should be simple enough that no technical background is needed to use it.
- **Reliability:** the same cleaning and encoding logic must be used at both training time and prediction time, so there's no mismatch between the two.
- **Portability:** the app should run on any machine with Python, PyTorch, and Streamlit installed — no GPU required.
- **Maintainability:** the code should be split into separate, modular pieces (preprocessing, model, prediction, database, UI) so it's easy to extend later.
- **Data persistence:** prediction history should stick around even after the app restarts, since it's stored in a local SQLite file.

4. Project Design and Implementation

4.1. Proposed Project Architecture

The system is built as a straightforward, linear pipeline rather than a complex distributed setup, since a single review needs to move through a fixed sequence of steps before a result comes out. A review typed by the user first passes through the preprocessing stage, where it is cleaned and standardised. It is then converted into a numeric sequence the model can actually read, and passed through the BiLSTM network, which studies the sentence in both directions to understand its overall tone. The output of this step is a raw score, which is turned into a clear label - Positive or Negative - along with a confidence value. This final result is shown to the user immediately and, at the same time, written to the SQLite database, which in turn feeds the Dashboard page with charts, filters, and history. This simple, step-by-step design keeps the system easy to follow, debug, and extend, since each stage has one clear job and hands its output to the next.

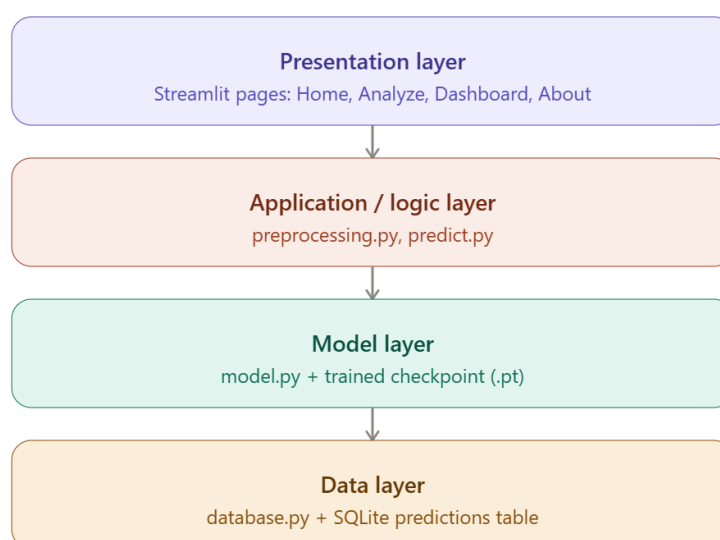


4.2. High Level Design

Rather than cramming everything into one big script, the system is split into four layers, each doing its own separate job. At the top is the Presentation Layer — this is simply what the user sees and clicks on, the Streamlit pages (Home, Analyze, Dashboard, About) that

take in the review text and show back the results. Just below it sits the Application/Logic Layer, which quietly does the heavy lifting: cleaning up the text and running it through the model to get a prediction. Underneath that is the Model Layer, home to the BiLSTM network

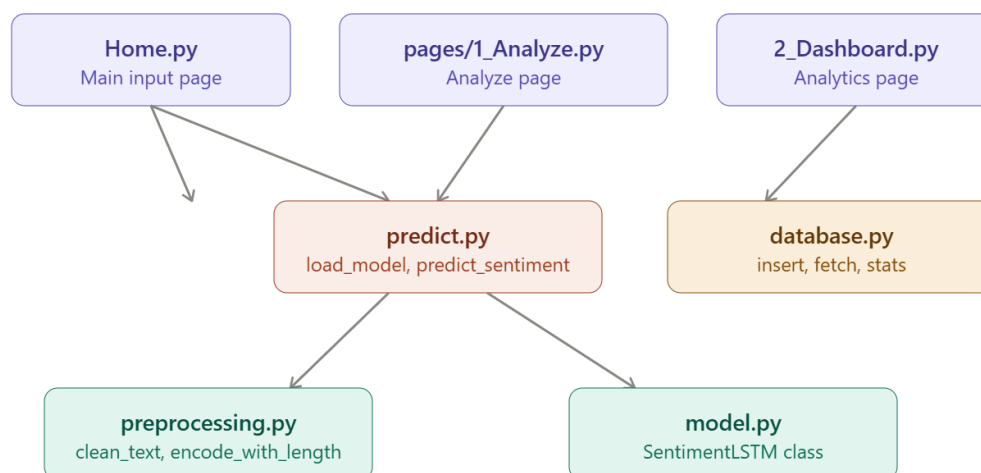
along with its saved weights and vocabulary — the part that actually "understands" the review. And at the bottom is the Data Layer, which looks after the SQLite database, keeping a record of every prediction so it can be pulled up again later. Breaking the system down this way keeps each layer focused on one job, makes it much easier to swap out or fix one part without disturbing the rest (say, replacing the model without touching the UI at all), and follows a structure that's fairly standard in how real applications tend to be built.



4.3.Detailed Design

Looking closer at how the pieces actually connect: the Home page and the Analyze page are where a review first enters the system, and both of them hand the text off to `predict.py`. This file doesn't do the work itself — instead, it leans on two other modules. It calls `preprocessing.py` to clean up the raw text (lower-casing it, stripping out HTML tags and URLs, turning punctuation into spaces) and turn it into a padded sequence of numbers the model can actually process. It then calls `model.py`, which holds the `SentimentLSTM` class — the embedding layer, the two-layer bidirectional LSTM, and

the final linear layer that produces a raw score. Once `predict.py` gets this score back, it applies a sigmoid function to turn it into a proper probability and decides on a label. From there, both the Home page and the Dashboard page talk to `database.py`, which handles saving new predictions and pulling up past ones. Every time a prediction is made, it gets written to the SQLite database through this module, and the Dashboard reads from the same place whenever it needs to build its charts, run a search, or export a CSV. Keeping things split this way — text cleaning, model logic, and storage all living in their own separate files — means each piece can be tested, fixed, or swapped out on its own without the rest of the code needing to change.



4.4.Dataset Description

Training data comes from the IMDb Large Movie Review Dataset, first put together by Maas et al. (2011) and now a standard benchmark for sentiment classification tasks. It holds 50,000 strongly opinionated movie reviews pulled from the Internet Movie Database, split evenly — 25,000 for training, 25,000 for testing — with another 50,000 unlabelled reviews available if needed for semi-supervised approaches. Only reviews with a clearly strong rating are labelled (7/10 or above counts as positive, 4/10 or below counts as negative), which keeps the task focused without being too easy. Both the training and test sets are perfectly balanced, with 12,500 positive and 12,500 negative reviews each.

4.5.Tools and Technologies Used for Implementation

Programming & Model Building

- Python 3 was used as the core language for the entire project, from data handling to deployment.
- PyTorch was the deep learning framework of choice, used to design, train, and run the BiLSTM sentiment model.

Web Interface

- Streamlit powers the entire front-end — it turns plain Python scripts into an interactive, multi-page web app without needing separate HTML/CSS/JavaScript code.

Data Handling & Visualization

- Pandas is used to organize and manipulate the prediction history in tabular form, especially for the Dashboard and CSV export.
- Altair generates the interactive charts on the Dashboard — the pie chart for sentiment breakdown and the line chart for daily trends.

Storage

- SQLite (accessed via Python's built-in sqlite3 module) provides lightweight, file-based storage for every prediction, requiring no separate database server.

Dataset

- The IMDb Large Movie Review Dataset (50,000 labelled reviews) serves as the training and evaluation data for the sentiment model.

Version Control

- Git and GitHub were used to track code changes and manage the project repository throughout development.

5. Result and Discussion

The developed sentiment-analysis system was evaluated based on the functional and non-functional requirements defined for the application. The implemented system accepts movie reviews as input, performs text preprocessing and vocabulary encoding, and uses a pretrained two-layer BiLSTM model to classify the review as positive or negative. The prediction is accompanied by a confidence score and is automatically stored in a local SQLite database. In addition, the system provides a dashboard for reviewing prediction statistics, searching and filtering historical predictions, and exporting the stored results.

5.1 Experimentation Environment Setup

The sentiment-analysis application was implemented as a Python-based web application using Streamlit. PyTorch was used for loading and executing the pretrained BiLSTM sentiment-classification model. SQLite was used as the local database for persistent storage of prediction history.

The model processes each review through text preprocessing, tokenization, vocabulary conversion, and fixed-length sequence preparation. Each input sequence is padded or truncated to a maximum length of 200 tokens before being passed to the embedding layer and two-layer BiLSTM network.

The major hardware and software components used during experimentation are summarized in Table 1.

Table 1. Experimentation environment

Component	Configuration/Technology
Programming Language	Python
Deep Learning Framework	PyTorch
Neural Network	Two-Layer BiLSTM
Text Representation	Tokenization and Embedding
Maximum Sequence Length	200 tokens
Activation Function	Sigmoid

Regularization	Dropout
Database	SQLite
Application Framework	Streamlit
Visualization	Streamlit charts and analytical components
Operating System	Windows
Development Environment	Visual Studio Code
Input	Product review text
Output	Positive/Negative sentiment and confidence

The trained model was integrated into the application so that a user can enter a review and obtain the sentiment prediction in real time. The same prediction is stored in the SQLite database, enabling the dashboard to maintain a historical record of analyzed reviews.

5.2 Functional Requirements Results Achieved

The developed system successfully implements the major functional requirements defined for the sentiment-analysis application. The user can submit a product review, obtain a sentiment prediction, view the confidence score, and examine previously analyzed reviews through the dashboard.

Table 2. Functional requirements and achieved results

Functional Requirement	Implementation	Result
Review Input	Text input interface	Successfully accepts product reviews
Text Preprocessing	Cleaning and normalization	Input is prepared for model processing
Tokenization	Conversion of text into tokens	Successfully performed
Vocabulary Conversion	Tokens converted into numerical indices	Successfully performed

Padding/Truncation	Sequence length fixed to 200 tokens	Successfully implemented
Sentiment Prediction	Two-layer BiLSTM model	Produces positive/negative prediction
Confidence Calculation	Sigmoid probability	Confidence value displayed
Prediction Storage	SQLite database	Prediction history maintained
Search/Filtering	Dashboard functionality	Stored results can be explored
Analytics	Charts and summary information	Historical sentiment trends can be visualized
CSV Export	Dashboard functionality	Prediction records can be exported

Prediction Results

For each submitted review, the system generates a binary sentiment prediction. The sigmoid output represents the probability associated with the predicted class. The result is then displayed to the user together with the corresponding confidence value.

For example, reviews expressing satisfaction with a product are expected to produce a positive prediction, whereas reviews expressing dissatisfaction are expected to produce a negative prediction. The confidence value provides an indication of how strongly the model supports the predicted class.

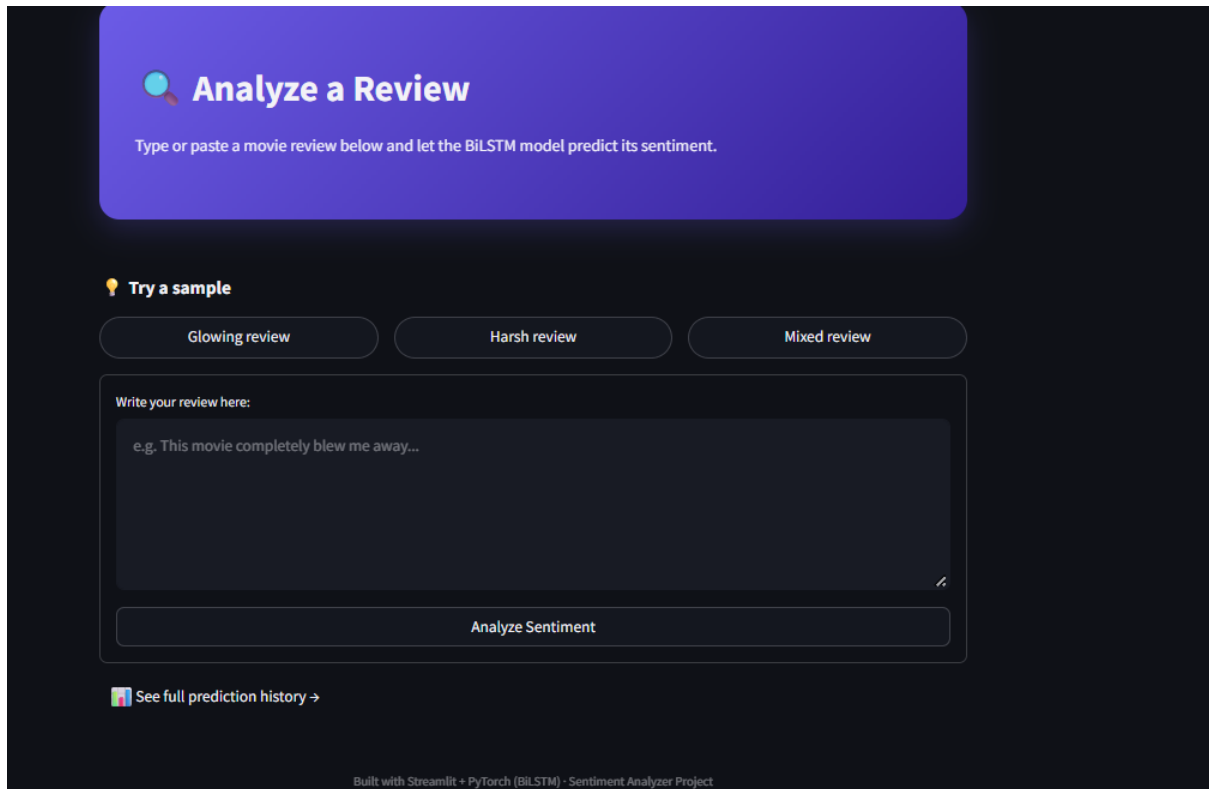
Table 3. Example prediction outputs

Review Type	Predicted Sentiment
Positive product feedback	Positive
Negative product feedback	Negative
Strongly positive review	Positive
Strongly negative review	Negative

The system therefore satisfies the primary functional objective of automatically analyzing review text and producing an interpretable sentiment prediction.

Graphical Results

The dashboard provides graphical representations of the stored prediction history. These visualizations allow users to observe the distribution of positive and negative reviews and identify sentiment trends within the analyzed dataset.



The screenshot shows a web interface titled "Analyze a Review" with a magnifying glass icon. Below the title, it says "Type or paste a movie review below and let the BiLSTM model predict its sentiment." There is a section labeled "Try a sample" with three buttons: "Glowing review", "Harsh review", and "Mixed review". Below these is a text input area with the placeholder "Write your review here:" and an example text "e.g. This movie completely blew me away...". At the bottom of the input area is a button labeled "Analyze Sentiment". Below the input area is a link "See full prediction history →". At the very bottom, it says "Built with Streamlit + PyTorch (BiLSTM) - Sentiment Analyzer Project".

Figure 1. Sentiment distribution of analyzed product reviews.

The sentiment-distribution graph provides a visual comparison between the number of positive and negative predictions. A larger proportion of positive predictions indicates that more of the analyzed reviews express favorable opinions, while a larger negative proportion indicates greater dissatisfaction among the analyzed reviews.

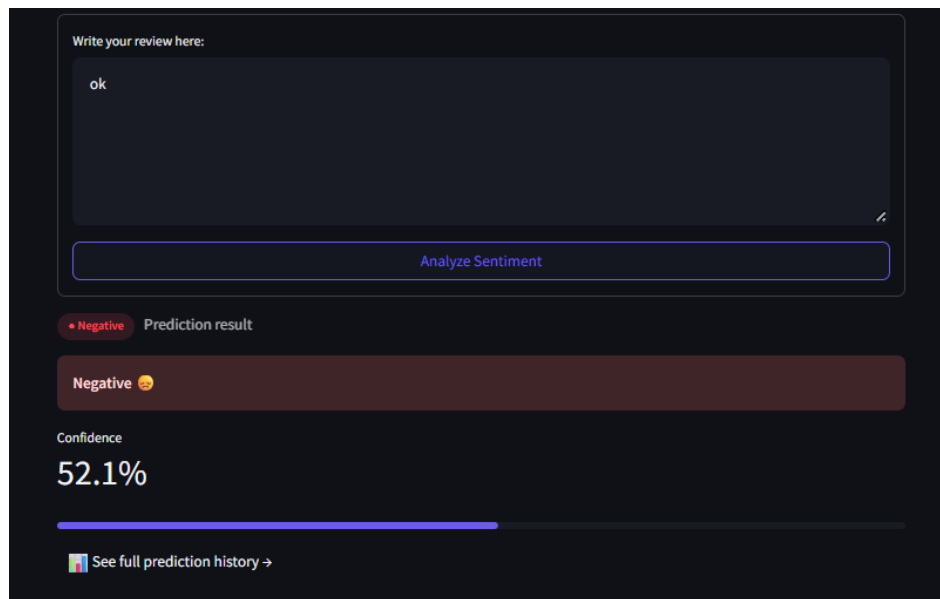


Figure 2. Confidence distribution of sentiment predictions.

The confidence visualization represents the distribution of model confidence across analyzed reviews. High-confidence predictions indicate that the model assigns a strong probability to the predicted sentiment class, whereas lower-confidence predictions may correspond to ambiguous or contextually difficult reviews.

5.3 Non-Functional Requirements Results Achieved

In addition to functional capabilities, the system was evaluated against important non-functional requirements such as usability, responsiveness, reliability, maintainability, and data persistence.

Table 4. Non-functional requirements and achieved results

Non-Functional Requirement	Result Achieved
Usability	Simple interface allows users to enter reviews and view predictions
Responsiveness	Prediction results are displayed after processing the submitted review
Reliability	The trained model consistently follows the defined prediction pipeline

Maintainability	The application is organized into model, preprocessing, database, and interface components
Data Persistence	SQLite maintains historical prediction records
Scalability	The framework can be extended to additional review datasets and model architectures
Interpretability	Sentiment class and confidence value are displayed to the user
Accessibility	Browser-based Streamlit interface can be used without a separate desktop application
Data Management	Historical predictions can be searched, filtered, analyzed, and exported

The results demonstrate that the proposed application is not limited to producing a single prediction. It also provides supporting facilities for storing and analyzing predictions, making it suitable for practical experimentation and demonstration.

5.4 Comparison of Results with Existing Works

The proposed framework was compared conceptually with approaches discussed in the literature survey. Traditional sentiment-analysis approaches commonly rely on manually engineered features and conventional machine-learning classifiers. Deep-learning approaches, in contrast, can learn useful representations directly from sequential text.

The proposed system uses a two-layer BiLSTM architecture with an embedding layer. The bidirectional structure enables the model to process contextual information from both preceding and succeeding words. This is particularly useful for sentiment analysis because the sentiment expressed by a word may depend on surrounding words.

5.5 Project GUI Snapshots

The developed Streamlit application provides an interactive graphical interface for sentiment prediction and historical analysis. The major GUI functionalities are described below.

5.5.1 Review Input Interface

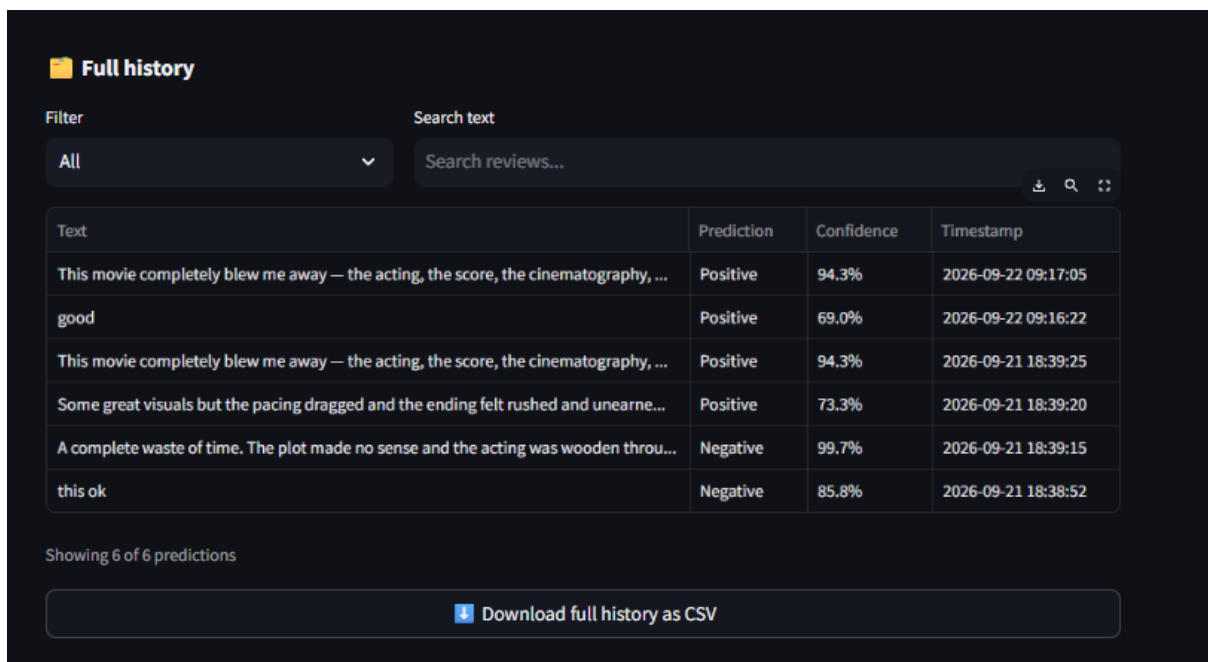
The review input section allows the user to enter a product review into the application. After submission, the text is passed through the preprocessing and prediction pipeline.

5.5.2 Sentiment Prediction Result

After processing the review, the application displays the predicted sentiment as either positive or negative. The corresponding confidence value is also presented to provide additional information about the prediction.

5.5.3 Prediction History

The prediction-history section displays previously analyzed reviews together with their predicted sentiment and associated information. The records are retrieved from the SQLite database.



The screenshot shows a web interface titled "Full history" with a dark theme. It includes a "Filter" dropdown set to "All" and a "Search text" input field. Below these is a table with four columns: "Text", "Prediction", "Confidence", and "Timestamp". The table contains six rows of data. At the bottom, it says "Showing 6 of 6 predictions" and has a button to "Download full history as CSV".

Text	Prediction	Confidence	Timestamp
This movie completely blew me away – the acting, the score, the cinematography, ...	Positive	94.3%	2026-09-22 09:17:05
good	Positive	69.0%	2026-09-22 09:16:22
This movie completely blew me away – the acting, the score, the cinematography, ...	Positive	94.3%	2026-09-21 18:39:25
Some great visuals but the pacing dragged and the ending felt rushed and unearne...	Positive	73.3%	2026-09-21 18:39:20
A complete waste of time. The plot made no sense and the acting was wooden throu...	Negative	99.7%	2026-09-21 18:39:15
this ok	Negative	85.8%	2026-09-21 18:38:52

Showing 6 of 6 predictions

Download full history as CSV

Figure 3. Historical prediction records stored in the system.

5.5.4 Dashboard and Analytics

The dashboard provides graphical summaries of the analyzed reviews. Users can examine sentiment distributions, prediction statistics, and historical trends through visual analytical .

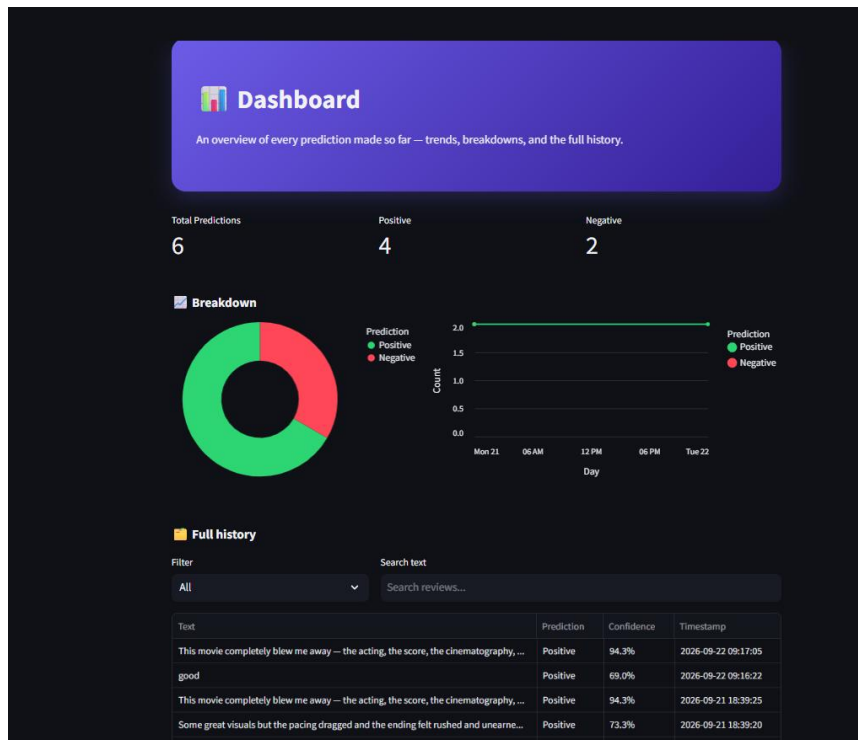


Figure 4. Sentiment-analysis dashboard and analytics.

5.5.5 Search, Filtering, and Export

The application provides facilities for searching and filtering prediction records. The stored results can also be exported in CSV format for further analysis.

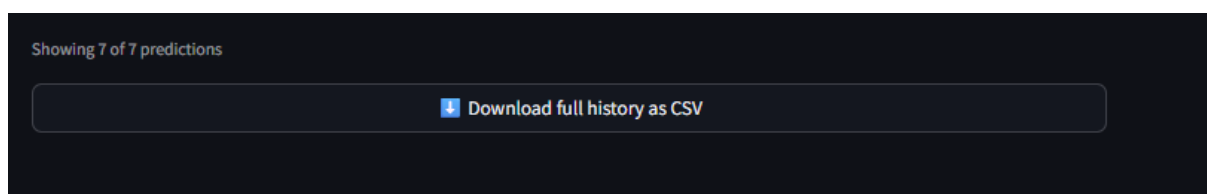


Figure 5. Search, filtering, and CSV export functionality.

The GUI demonstrates that the proposed framework provides an end-to-end environment in which users can submit reviews, obtain predictions, inspect confidence values, and analyze historical sentiment results.

5.6 Societal Impact of the Project

Sentiment analysis of product reviews has practical value for both consumers and organizations. Large numbers of online reviews can contain useful information regarding customer satisfaction, product quality, service experience, and common complaints. However, manually examining large volumes of reviews is time-consuming.

The proposed framework can assist in automatically organizing review information according to sentiment. Businesses can potentially use such analysis to identify frequently expressed positive and negative opinions and understand customer feedback at a broader level. Consumers can also benefit indirectly when organizations use review insights to improve product quality and services.

The system can further support data-driven decision-making by converting unstructured textual feedback into structured sentiment information. However, automated sentiment predictions should be treated as analytical assistance rather than a complete replacement for human interpretation, particularly for reviews containing sarcasm, mixed opinions, domain-specific terminology, or ambiguous language.

5.7 SDG Mapped

The proposed project can be associated with the **United Nations Sustainable Development Goal (SDG) 9: Industry, Innovation and Infrastructure**, particularly through the application of artificial intelligence and deep-learning technologies to process large-scale textual information.

The system also has relevance to **SDG 12: Responsible Consumption and Production**, as product-review analysis can help organizations understand consumer feedback, identify recurring product-related issues, and support improvements in products and services.

Table 6. SDG mapping of the proposed project

SDG	Goal	Relevance to the Project
SDG 9	Industry, Innovation and Infrastructure	Applies deep learning and NLP technologies to develop an intelligent sentiment-analysis system
SDG 12	Responsible Consumption and Production	Supports analysis of consumer feedback that can contribute to product and service improvement
SDG 8	Decent Work and Economic Growth	AI-assisted analysis can improve the efficiency of review processing and business intelligence activities

Overall, the experimental results demonstrate that the proposed sentiment-analysis framework provides an integrated solution for automated product-review classification. The combination of preprocessing, embedding, two-layer BiLSTM classification, confidence estimation, SQLite-based storage, and dashboard analytics provides a complete workflow from raw review input to analytical output.

Conclusion and Future Scope

The Movie Review Sentiment Analyzer demonstrates how deep learning and natural language processing can be combined to build a practical sentiment analysis system. The BiLSTM model successfully captures contextual information from review text by processing it in both forward and backward directions, achieving an accuracy of approximately 83% on the IMDb test dataset. The integration of the trained model with a Streamlit web application makes sentiment prediction simple and accessible to users in real time. In addition, storing prediction results in an SQLite database and providing dashboard features such as visualization, filtering, searching, and CSV export makes the system useful beyond basic classification. Overall, the project shows the effectiveness of BiLSTM-based sentiment analysis for understanding customer opinions and demonstrates how machine learning models can be transformed into interactive applications. Future improvements could include using larger and more diverse datasets, hyperparameter optimization, multilingual sentiment

analysis, and transformer-based models such as BERT to achieve higher accuracy and better contextual understanding.

References

- [1] “Sentiment analysis of movie reviews: A Flask application using CNN with RoBERTa embeddings,” *ScienceDirect*, vol. 7, 2025, Art. no. 200192, doi: 10.1016/j.sasc.2025.200192.
- [2] Greeshma and Simon, “Sentiment analysis using BiGRU with attention and GloVe,” 2024.
- [3] A. R. Krishna, A. S. Vardhini, R. P. Singh, and S. Kanchan, “Efficient Sentiment Analysis on IMDb Movie Reviews with Synonym Augmentation and Global Average Pooling,” in *Proc. 3rd Int. Conf. Artificial Intelligence for Internet of Things (AIIoT)*, 2024, doi: 10.1109/AIIoT58432.2024.10574775.
- [4] Palomo *et al.*, “Comparative evaluation of BERT, RoBERTa and XLNet for sentiment analysis,” 2024.
- [5] Atandoh *et al.*, “PEW-MCAB: A scalable deep learning framework for sentiment analysis,” 2024.
- [6] “BiLSTM and BiLSTM-CNN based sentiment analysis of drug reviews,” *NCBI/PMC*, 2024.
- [7] K. Gill, V. Anand, R. Chauhan, and A. Choudhary, “CNN, LSTM, and Bi-LSTM based Self-Attention Model Classification for User Review Sentiment Analysis,” in *Proc. 3rd Int. Conf. Smart Generation Computing, Communication and Networking (SMART GENCON)*, 2023, doi: 10.1109/SMARTGENCON60755.2023.10442498.
- [8] S. K. Singh and N. Singla, “Sentiment Analysis on IMDB Review Dataset,” *Journal of Computers, Mechanical and Management*, vol. 2, no. 6, pp. 18–29, 2023, doi: 10.57159/gadl.jcmm.2.6.230108.
- [9] Ansari *et al.*, “Word2Vec vs. BERT for sentiment analysis,” arXiv:2304.08369, 2023.
- [10] “A survey of sentiment analysis methods,” arXiv:2311.11250, 2023.
- [11] Jiang, Hu, and Jiang, “Sentiment analysis of movie reviews using Word2Vec and LSTM,” 2022.
- [12] Chakraborty *et al.*, “Word embedding techniques for sentiment analysis,” arXiv:2104.10662, 2021.

- [13] Z. Hameed and B. Garcia-Zapirain, “Sentiment Classification Using a Single-Layered BiLSTM Model,” *IEEE Access*, vol. 8, pp. 73992–74001, 2020, doi: 10.1109/ACCESS.2020.2988550.
- [14] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [15] M. Rezaeinia *et al.*, “Sentiment analysis based on improved word vectors,” 2017.
- [16] M. Giatsoglou *et al.*, “Sentiment analysis leveraging emotions and word embeddings,” *Expert Systems with Applications*, 2017.
- [17] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [18] J. Pennington, R. Socher, and C. D. Manning, “GloVe: Global Vectors for Word Representation,” in *Proc. EMNLP*, 2014, pp. 1532–1543.
- [19] Y. Kim, “Convolutional Neural Networks for Sentence Classification,” in *Proc. EMNLP*, 2014, pp. 1746–1751.
- [20] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013.